Name : P. Yaswanth

Reg No : 192524167

Experiment: A* (A-Star) Search Algorithm

Aim

To implement the **A*** (A-Star) **Search Algorithm** using Python to find the shortest path between a start node and a goal node.

Objectives

- To understand the working of the A* Search Algorithm.
- To implement the A* algorithm using Python.
- To find the optimal path between the start and goal nodes.
- To use heuristic values to improve search efficiency.
- To understand the application of informed search techniques in Artificial Intelligence.

Algorithm

1. Initialize the **open list** with the start node and an empty **closed list**.
2. Assign the start node with:
    - **g(n)** = 0 (cost from start)
    - **h(n)** = heuristic estimate to the goal
    - **f(n) = g(n) + h(n)**
3. Select the node with the lowest **f(n)** value from the open list.
4. If the selected node is the goal node, display the shortest path.
5. Otherwise, expand its neighboring nodes.
6. Calculate the **g(n), h(n),** and **f(n)** values for each neighbor.
7. Repeat the process until the goal node is reached or no path exists.

Python Program

```
import heapq
```

```
# Graph representation
graph = {
   'A': [('B', 1), ('C', 3)],
   'B': [('D', 3), ('E', 1)],
   'C': [('F', 5)],
   'D': [('G', 2)],
   'E': [('G', 1)],
   'F': [('G', 2)],
   'G': []
}

# Heuristic values
heuristic = {
   'A': 5,
   'B': 4,
   'C': 4,
   'D': 2,
   'E': 1,
   'F': 2,
   'G': 0
}

def a_star(start, goal):
   open_list = []
   heapq.heappush(open_list, (0, start))
```

```
g_cost = {start: 0}
parent = {start: None}

while open_list:
    _, current = heapq.heappop(open_list)

    if current == goal:
        path = []
        while current:
            path.append(current)
            current = parent[current]
        path.reverse()

        print("Shortest Path:", " -> ".join(path))
        print("Total Cost:", g_cost[goal])
        return

    for neighbor, cost in graph[current]:
        new_cost = g_cost[current] + cost

        if neighbor not in g_cost or new_cost < g_cost[neighbor]:
            g_cost[neighbor] = new_cost
            f_cost = new_cost + heuristic[neighbor]
            heapq.heappush(open_list, (f_cost, neighbor))
            parent[neighbor] = current
```

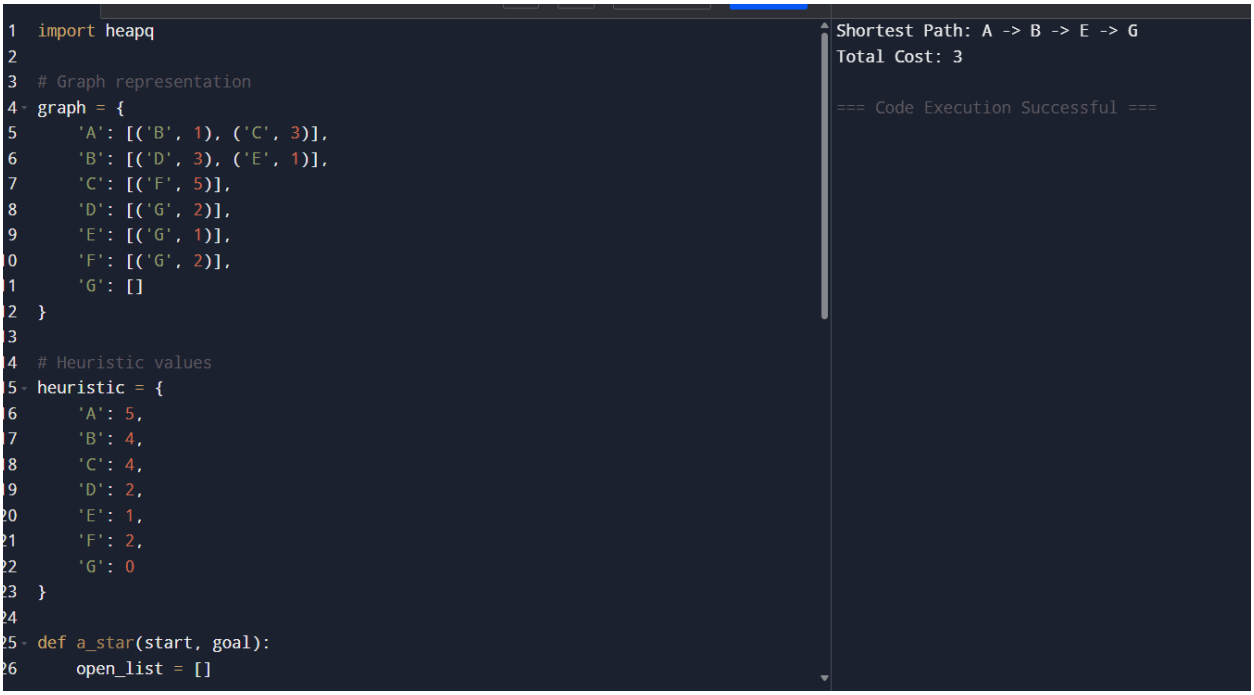
```
    print("No path found.")


# Driver Code

a_star('A', 'G')
```

Sample Output

Shortest Path: A -> B -> E -> G

Total Cost: 3

Out Put



Result

The A* Search Algorithm was successfully implemented using Python. The program found the optimal path from the start node to the goal node by considering both the actual path cost and the heuristic estimate.

Conclusion

The A* (A-Star) algorithm is an **informed search algorithm** that efficiently finds the shortest path by evaluating both the actual cost **g(n)** and the estimated cost **h(n)**. It guarantees an optimal solution when the heuristic is admissible and is widely used in

Artificial Intelligence applications such as pathfinding, robotics, navigation systems, and game development.